



LAB REPORT- 01

Department of Computer Science & Engineering (CSE)

Faculty of Science and Information Technology (FSIT)

Summer 2026

Course Title : Computer Graphics Lab
Course Code : CSE412
Course Credit : 1.5
Date of Submission : June 13 , 2026
Topic Covered : Drawing 8 Stars with RGB
Coloring Scheme Using OpenGL

Submitted By

Shafin Ahmed

232-15-184

65_M1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

Experiment No: 01

Experiment Name: Drawing 8 Stars with RGB Coloring Scheme Using OpenGL

Project Name: eightstars

Introduction

Overview

In this experiment, I have implemented a simple computer graphics application using OpenGL and GLUT in C. The objective of this experiment has been to draw eight star-shaped objects using different RGB color combinations while working within a customized coordinate system.

I have modified the X-axis and Y-axis ranges from 0 to 300 and adjusted the window size to 700 × 700 pixels. Additionally, I have converted the color values from the standard OpenGL `glColor3f()` format into the RGB (0–255) format and used multiple color combinations to make the stars visually appealing.

Topics Covered

- OpenGL Graphics Programming
 - GLUT Window Management
 - RGB Coloring Scheme
 - Coordinate System Transformation
 - Polygon Drawing
 - 2D Graphics Rendering
-

Environment / Platform

I have performed this experiment using the following environment:

- Programming Language: C
- Graphics Library: OpenGL (GLUT)

- IDE: CodeBlocks
- Operating System: Windows
- Project Title: eightstars
- Project Filename: eightstars.cbp

Project Details

- **Project Title:** eightstars
 - **Folder:** LabReport01
 - **Project Filename:** eightstars.cbp
 - **Resulting Filename:** LabReport01\eightstars\eightstars.cbp
-

Task

Description

The task was to create a graphical application that displays eight stars using OpenGL polygons. The stars were drawn at different positions within the coordinate system and assigned different RGB colors.

The following requirements were completed:

- Changed the X-axis range to 0–300.
 - Changed the Y-axis range to 0–300.
 - Drew eight stars using OpenGL polygons.
 - Converted color values from OpenGL's normalized format to RGB (0–255) format.
 - Set the window size to 700 × 700 pixels.
 - Used different colors for each star.
-

Source Code

```
//Shafin Ahmed 232-15-184 65-M1 (Lab 03)
#include <GL/gl.h>
#include <GL/glut.h>
/* everything inside the function display() appears on the
screen. */
void display(void)
{
/* glClear() clears the screen using the background color specified earlier */
/* GL_COLOR_BUFFER_BIT means clear the color buffer (screen) */
glClear (GL_COLOR_BUFFER_BIT);
/* basically erase all previously drawn colors and start fresh. */
///1st body //Pink
```

```

float Red=255;
float Green=3;
float Blue=200;
glColor3f (Red/255,Green/255, Blue/255); /* glColor3f(R, G, B) */
// glColor3f(1,0,0); // red
// glColor3f(0,1,0); // green
// glColor3f(0,0,1); // blue
// glColor3f(1,1,1); // white
// glColor3f(0,0,0); // black
glBegin(GL_POLYGON);
/* OpenGL connects all vertices in order and
fills the shape */
glVertex3f (153, 142, 0.0);
glVertex3f (156, 134, 0.0);
glVertex3f (159, 142, 0.0);
glVertex3f (167, 145, 0.0);
glVertex3f (159, 148, 0.0);
glVertex3f (156, 156, 0.0);
glVertex3f (153, 148, 0.0);
glVertex3f (145, 145, 0.0);
glEnd();
///5th body //Komola
Red=245;
Green=73;
Blue=39;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (31, 142, 0.0);
glVertex3f (34, 134, 0.0);
glVertex3f (37, 142, 0.0);
glVertex3f (45, 145, 0.0);
glVertex3f (37, 148, 0.0);
glVertex3f (34, 156, 0.0);
glVertex3f (31, 148, 0.0);
glVertex3f (23, 145, 0.0);
glEnd();
///2nd body //Beguni
Red=152;
Green=23;
Blue=227;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (131, 142, 0.0);
glVertex3f (134, 134, 0.0);
glVertex3f (137, 142, 0.0);
glVertex3f (145, 145, 0.0);
glVertex3f (137, 148, 0.0);
glVertex3f (134, 156, 0.0);
glVertex3f (131, 148, 0.0);
glVertex3f (123, 145, 0.0);
glEnd();
///6th body //Lal
Red=227;
Green=23;

```

```

Blue=37;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
/* OpenGL connects all vertices in order and
fills the shape */
glVertex3f (253, 142, 0.0);
glVertex3f (256, 134, 0.0);
glVertex3f (259, 142, 0.0);
glVertex3f (267, 145, 0.0);
glVertex3f (259, 148, 0.0);
glVertex3f (256, 156, 0.0);
glVertex3f (253, 148, 0.0);
glVertex3f (245, 145, 0.0);
glEnd();
///3rd body //Sobuj
Red=250;
Green=233;
Blue=22;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (142, 153, 0.0);
glVertex3f (145, 145, 0.0);
glVertex3f (148, 153, 0.0);
glVertex3f (156, 156, 0.0);
glVertex3f (148, 159, 0.0);
glVertex3f (145, 167, 0.0);
glVertex3f (142, 159, 0.0);
glVertex3f (134, 156, 0.0);
glEnd();
///7th body //Sobuj
Red=167;
Green=250;
Blue=22;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (142, 253, 0.0);
glVertex3f (145, 245, 0.0);
glVertex3f (148, 253, 0.0);
glVertex3f (156, 256, 0.0);
glVertex3f (148, 259, 0.0);
glVertex3f (145, 267, 0.0);
glVertex3f (142, 259, 0.0);
glVertex3f (134, 256, 0.0);
glEnd();
///4th body //Brown
Red=176;
Green=96;
Blue=53;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (142, 131, 0.0);
glVertex3f (145, 123, 0.0);
glVertex3f (148, 131, 0.0);
glVertex3f (156, 134, 0.0);

```

```

glVertex3f (148, 137, 0.0);
glVertex3f (145, 145, 0.0);
glVertex3f (142, 137, 0.0);
glVertex3f (134, 134, 0.0);
glEnd();
///8th body //Nil
Red=31;
Green=126;
Blue=237;
glColor3f (Red/255,Green/255, Blue/255);
glBegin(GL_POLYGON);
glVertex3f (142, 51, 0.0);
glVertex3f (145, 43, 0.0);
glVertex3f (148, 51, 0.0);
glVertex3f (156, 54, 0.0);
glVertex3f (148, 57, 0.0);
glVertex3f (145, 65, 0.0);
glVertex3f (142, 57, 0.0);
glVertex3f (134, 54, 0.0);
glEnd();
/* start processing buffered OpenGL routines */
glFlush ();
}
void init (void)
{
/* clearing (background) color / sets it to black */
glClearColor (0.0, 0.0, 0.0, 0.0);
/* initialize viewing values or configures the camera */
glMatrixMode(GL_PROJECTION);
glLoadIdentity(); //resets any previous transformation
glOrtho(0.0, 300.0, 0.0, 300.0,-1.0, 1.0);
// creates 2D coordinate space
/* This treats the window as a graph */
}
/*
* Declare initial window size, position, and display mode
* (single buffer and RGBA). Open window with "hello"
* in its title bar. Call initialization routines.
* Register callback function to display graphics.
* Enter main loop and process events.
*/
int main(int argc, char** argv)
{
glutInit(&argc, argv); // glut initialization
glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
// single buffering and RGB colors
glutInitWindowSize (700, 700);
// size 500x500
glutInitWindowPosition (100, 0);
// 100 pixels from left
// 100 pixels from top
glutCreateWindow ("Star Designing");
init ();
glutDisplayFunc(display);

```

```
glutMainLoop();  
/* without this line the window would immediately close */  
return 0; /* ISO C requires main to return int. */  
}
```

Code Description

Header Files

I have included the following header files:

- `GL/gl.h` for OpenGL functions.
- `GL/glut.h` for GLUT window management and event handling.

These libraries provide the necessary functions to create windows, draw graphics, and manage rendering operations.

Display Function

The `display()` function is responsible for rendering all graphical objects on the screen.

Inside this function:

- The screen is cleared using `glClear()`.
- Different RGB color combinations are assigned using `glColor3f()`.
- Eight star shapes are created using `GL_POLYGON`.
- Each star is drawn by specifying multiple vertices through `glVertex3f()`.
- The rendering process is completed using `glFlush()`.

RGB Color Conversion

Instead of directly using normalized values between 0 and 1, I have used RGB values ranging from 0 to 255 and converted them as follows:

RGB Value / 255

Example:

```
float Red = 255;  
float Green = 3;  
float Blue = 200;
```

```
glColor3f(Red/255, Green/255, Blue/255);
```

This method makes color selection easier and more intuitive.

Initialization Function

The `init()` function initializes the graphics environment.

I have:

- Set the background color to black.
- Configured the projection matrix.
- Created a 2D coordinate system using:

```
glOrtho(0.0, 300.0, 0.0, 300.0, -1.0, 1.0);
```

This allows the window to behave like a graph where coordinates range from 0 to 300 on both axes.

Main Function

The `main()` function performs the following tasks:

- Initializes GLUT.
- Sets RGB display mode.
- Creates a window of size 700 × 700.
- Registers the display callback function.
- Starts the OpenGL event loop.

Input & Output

```
main.cpp x
1 //Shafin Ahmed 232-15-184 65-M1 (Lab 03)
2 #include <GL/gl.h>
3 #include <GL/glut.h>
4 /* everything inside the function display() appears on the
5 screen. */
6 void display(void)
7 {
8     /* glClear() clears the screen using the background color specified earlier */
9     /* GL_COLOR_BUFFER_BIT means clear the color buffer (screen) */
10    glClear (GL_COLOR_BUFFER_BIT);
11    /* basically erase all previously drawn colors and start fresh. */
12    //1st body //Pink
13    float Red=255;
14    float Green=3;
15    float Blue=200;
16    glColor3f (Red/255,Green/255, Blue/255); /* glColor3f(R, G, B) */
17    // glColor3f(1,0,0); // red
18    // glColor3f(0,1,0); // green
19    // glColor3f(0,0,1); // blue
20    // glColor3f(1,1,1); // white
21    // glColor3f(0,0,0); // black
22    glBegin(GL_POLYGON);
23    /* OpenGL connects all vertices in order and
24    fills the shape */
25    glVertex3f (153, 142, 0.0);
26    glVertex3f (156, 134, 0.0);
27    glVertex3f (159, 142, 0.0);
28    glVertex3f (167, 145, 0.0);
29    glVertex3f (159, 148, 0.0);
30    glVertex3f (156, 156, 0.0);
31    glVertex3f (153, 148, 0.0);
32    glVertex3f (145, 145, 0.0);
```



```

main.cpp x
31 glVertex3f (153, 148, 0.0);
32 glVertex3f (145, 145, 0.0);
33 glEnd();
34 ///5th body //Komola
35 Red=245;
36 Green=73;
37 Blue=39;
38 glColor3f (Red/255,Green/255, Blue/255);
39 glBegin(GL_POLYGON);
40 glVertex3f (31, 142, 0.0);
41 glVertex3f (34, 134, 0.0);
42 glVertex3f (37, 142, 0.0);
43 glVertex3f (45, 145, 0.0);
44 glVertex3f (37, 148, 0.0);
45 glVertex3f (34, 156, 0.0);
46 glVertex3f (31, 148, 0.0);
47 glVertex3f (23, 145, 0.0);
48 glEnd();
49 ///2nd body //Beguni
50 Red=152;
51 Green=23;
52 Blue=227;
53 glColor3f (Red/255,Green/255, Blue/255);
54 glBegin(GL_POLYGON);
55 glVertex3f (131, 142, 0.0);
56 glVertex3f (134, 134, 0.0);
57 glVertex3f (137, 142, 0.0);
58 glVertex3f (145, 145, 0.0);
59 glVertex3f (137, 148, 0.0);
60 glVertex3f (134, 156, 0.0);
61 glVertex3f (131, 148, 0.0);
62 glVertex3f (123, 145, 0.0);

```

```

main.cpp x
61 glVertex3f (131, 148, 0.0);
62 glVertex3f (123, 145, 0.0);
63 glEnd();
64 ///6th body //Lal
65 Red=227;
66 Green=23;
67 Blue=37;
68 glColor3f (Red/255,Green/255, Blue/255);
69 glBegin(GL_POLYGON);
70 /* OpenGL connects all vertices in order and
71 fills the shape */
72 glVertex3f (253, 142, 0.0);
73 glVertex3f (256, 134, 0.0);
74 glVertex3f (259, 142, 0.0);
75 glVertex3f (267, 145, 0.0);
76 glVertex3f (259, 148, 0.0);
77 glVertex3f (256, 156, 0.0);
78 glVertex3f (253, 148, 0.0);
79 glVertex3f (245, 145, 0.0);
80 glEnd();
81 ///3rd body //Sobuj
82 Red=250;
83 Green=233;
84 Blue=22;
85 glColor3f (Red/255,Green/255, Blue/255);
86 glBegin(GL_POLYGON);
87 glVertex3f (142, 153, 0.0);
88 glVertex3f (145, 145, 0.0);
89 glVertex3f (148, 153, 0.0);
90 glVertex3f (156, 156, 0.0);
91 glVertex3f (148, 159, 0.0);
92 glVertex3f (145, 167, 0.0);

```

```

main.cpp x
91 glVertex3f (148, 159, 0.0);
92 glVertex3f (145, 167, 0.0);
93 glVertex3f (142, 159, 0.0);
94 glVertex3f (134, 156, 0.0);
95 glEnd();
96 ///7th body //Sobuj
97 Red=167;
98 Green=250;
99 Blue=22;
100 glColor3f (Red/255,Green/255, Blue/255);
101 glBegin(GL_POLYGON);
102 glVertex3f (142, 253, 0.0);
103 glVertex3f (145, 245, 0.0);
104 glVertex3f (148, 253, 0.0);
105 glVertex3f (156, 256, 0.0);
106 glVertex3f (148, 259, 0.0);
107 glVertex3f (145, 267, 0.0);
108 glVertex3f (142, 259, 0.0);
109 glVertex3f (134, 256, 0.0);
110 glEnd();
111 ///4th body //Brown
112 Red=176;
113 Green=96;
114 Blue=53;
115 glColor3f (Red/255,Green/255, Blue/255);
116 glBegin(GL_POLYGON);
117 glVertex3f (142, 131, 0.0);
118 glVertex3f (145, 123, 0.0);
119 glVertex3f (148, 131, 0.0);
120 glVertex3f (156, 134, 0.0);
121 glVertex3f (148, 137, 0.0);
122 glVertex3f (145, 145, 0.0);

```

```
main.cpp x
121 glVertex3f (148, 137, 0.0);
122 glVertex3f (145, 145, 0.0);
123 glVertex3f (142, 137, 0.0);
124 glVertex3f (134, 134, 0.0);
125 glEnd();
126 //8th body //Nil
127 Red=31;
128 Green=126;
129 Blue=237;
130 glColor3f (Red/255,Green/255, Blue/255);
131 glBegin(GL_POLYGON);
132 glVertex3f (142, 51, 0.0);
133 glVertex3f (145, 43, 0.0);
134 glVertex3f (148, 51, 0.0);
135 glVertex3f (156, 54, 0.0);
136 glVertex3f (148, 57, 0.0);
137 glVertex3f (145, 65, 0.0);
138 glVertex3f (142, 57, 0.0);
139 glVertex3f (134, 54, 0.0);
140 glEnd();
141 /* start processing buffered OpenGL routines */
142 glFlush ();
143 }
144 void init (void)
145 {
146 /* clearing (background) color / sets it to black */
147 glClearColor (0.0, 0.0, 0.0, 0.0);
148 /* initialize viewing values or configures the camera */
149 glMatrixMode(GL_PROJECTION);
150 glLoadIdentity(); //resets any previous transformation
151 glOrtho(0.0, 300.0, 0.0, 300.0,-1.0, 1.0);
152 // creates 2D coordinate space
153 /* This treats the window as a graph */
154 }
155 /*
156 * Declare initial window size, position, and display mode
157 * (single buffer and RGBA). Open window with "hello"
158 * in its title bar. Call initialization routines.
159 * Register callback function to display graphics.
160 * Enter main loop and process events.
161 */
162 int main(int argc, char** argv)
163 {
164 glutInit(&argc, argv); // glut initialization
165 glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
166 // single buffering and RGB colors
167 glutInitWindowSize (700, 700);
168 // size 500x500
169 glutInitWindowPosition (100, 0);
170 // 100 pixels from left
171 // 100 pixels from top
172 glutCreateWindow ("Star Designing");
173 init ();
174 glutDisplayFunc(display);
175 glutMainLoop();
176 /* without this line the window would immediately close */
177 return 0; /* ISO C requires main to return int. */
178 }
179
```

```
main.cpp x
148 /* initialize viewing values or configures the camera */
149 glMatrixMode(GL_PROJECTION);
150 glLoadIdentity(); //resets any previous transformation
151 glOrtho(0.0, 300.0, 0.0, 300.0,-1.0, 1.0);
152 // creates 2D coordinate space
153 /* This treats the window as a graph */
154 }
155 /*
156 * Declare initial window size, position, and display mode
157 * (single buffer and RGBA). Open window with "hello"
158 * in its title bar. Call initialization routines.
159 * Register callback function to display graphics.
160 * Enter main loop and process events.
161 */
162 int main(int argc, char** argv)
163 {
164 glutInit(&argc, argv); // glut initialization
165 glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
166 // single buffering and RGB colors
167 glutInitWindowSize (700, 700);
168 // size 500x500
169 glutInitWindowPosition (100, 0);
170 // 100 pixels from left
171 // 100 pixels from top
172 glutCreateWindow ("Star Designing");
173 init ();
174 glutDisplayFunc(display);
175 glutMainLoop();
176 /* without this line the window would immediately close */
177 return 0; /* ISO C requires main to return int. */
178 }
179
```

Figure 1: Source Code in CodeBlocks

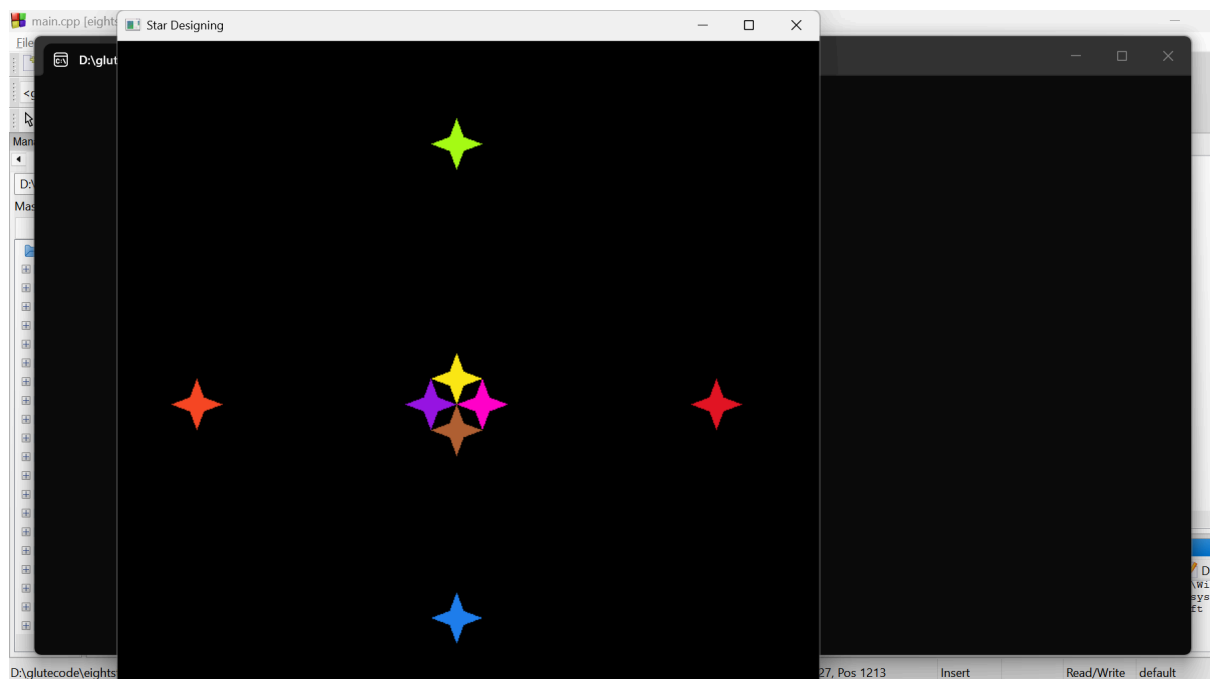


Figure 2: Program Output Window

Discussion

Reflection

Through this experiment, I have learned the fundamentals of OpenGL graphics programming. I have understood how graphical objects can be created using polygons and how colors can be applied using RGB values.

I have also learned how coordinate systems work in OpenGL and how window properties can be customized to create graphical applications.

This experiment has strengthened my understanding of basic computer graphics concepts such as rendering, color management, coordinate mapping, and object positioning.

Challenges Faced

Initially, positioning multiple stars within the coordinate space required careful adjustment of vertex coordinates. I also had to ensure that the stars remained visible and properly aligned after changing the coordinate system from the default settings.

Another challenge was converting RGB values into the normalized format required by OpenGL. This issue was resolved by dividing each RGB component by 255 before passing it to `glColor3f()`.

Conclusion

In this experiment, I have successfully implemented an OpenGL application that draws eight stars using different RGB color combinations. I have modified the coordinate system, customized the window size, and applied color conversion techniques to achieve the required output.

This experiment has provided practical experience in OpenGL programming and enhanced my understanding of fundamental computer graphics concepts that are essential for developing graphical applications.